A Comprehensive analysis of the CBP 2025 Branch Predictors entries: LVCP, RVA-Toru, and TAGE-SC 2025

Saumya Patel

March 26, 2026

Contents

1 Introduction

In the previous assignment, I evaluated the following branch predictors:

- Load Value Correlator Predictor (LVCP)
- Register-Value-Aware Branch Predictor (RVA-Toru)
- TAGE-SC 2025 André Seznec Predictor
- TAGE-SC-L Alberto Ros Predictor

In this report, I will continue my analysis of these predictors by performing a similar analysis of the remaining CBP2025 entries, namely

- MultiPerspective Perceptron Predictor (MPP) by Jimenez, et al.
- Programming Idiom Predictor (PIP)
- Branch Prediction via Load Value Prediction (BALL) by Jun Fan
- The Bullseye Predictor by Emet Behrendt, et al.

I will do a similar analysis of these predictors as I did for the first four and then I will compare all 8 predictors together to draw a final conclusion and rank them based on their performance.

1.1 The Oracle Predictor

The Oracle predictor is a theoretical construct that represents the best possible branch predictor that is not ahead of time. The point of this predictor is to check the maximum possible performance of a branch predictor on the given traces. It is important to note that the Oracle predictor is not a real predictor and cannot be implemented in hardware, but it serves as a useful benchmark for evaluating the performance of other predictors. More implementation details about the Oracle predictor can be found in the later sections of this report.

2 Methodology

The methodology for evaluating the predictor is the same as the one used in the previous assignment. I will be using the CBP2025 traces to evaluate the performance of the predictors. I will be using the same metrics as before, namely

- Branches Per Cycles (BrPerCyc)
- Misprediction Branches Per Cycles (MispBrPerCyc)
- Misprediction Rate (MR)
- Misprediction Rate Per Kilo Instructions (MPKI)
- Wrong Path Cycles (CycWP)
- Wrong Path Cycles Per Kilo Instructions (CycWPKI)

Furthermore, I also compared the predictors against each other to draw a final conclusion about their performance. The metrics that I used were,

- **Delta Misprediction Per Kilo Instructions:** $\Delta_{MPKI} = \text{MPKI}_A - \text{MPKI}_B$
  where A and B are two predictors being compared. This metric gives us a direct comparison of the misprediction rates of the two predictors.

3 MultiPerspective Perceptron Predictor (MPP)

3.1 Details

The Paper argues that traditional predictors look at branch history from two angles, the global and the local history. Jimenez argues that there are many useful "perspectives" on the same history. He introduces the MPP which is essentially a hashed perceptron that combines many of these perspectives together. The new features that this predictor introduces are the following:

- **MODHIST (Modulo History):** The author describes that he noticed a problem that he called ***branch misalignment*** i.e. if branch B sometimes appears in the history and some does not, then the predictor will have a hard time learning the pattern, because the same outcome sequence would land at different positions in the history register. This problem has seemed to break traditional and hashed predictors. To solve this problem, the author introduces MODHIST which essentially only records branches whose addresses are divisible by some modulos, filtering out the "misaligned" branches and thus making the history more consistent.

- **RECENCYPOS:** It tracks a stack of recently executed branches using LRU policy. knowing where the branch is in this stack can be useful to correlate the outcome of the branch with its recency.

- **BLURRYPATH:** It records coarser memory regions instead of exact addresses. A "coarser" memory region is defined as regions of memory who have the first N bits of their address the same. For example, if you take the address *0x7FFF4A28* and right shift it by 4 bits you would get *0x7FFF4A*, which would mean that every address that only differs in those last 4 bits would be considered the same "blurry" address.

  You would want to track these "blurry" addresses because of 2 reasons, **first**, they have less noise. If your program takes a slightly different path through the same general code region, precise history says "completely different." Blurry history says "same region, close enough." This can actually generalize better because you're capturing the structural shape of execution rather than its exact fingerprint. **Second**, compression. Fewer distinct values means less aliasing pressure in the hash tables which means that different histories are less likely to collide on the same table entry.

  MPP goes one step further, it only shifts a new region into the history when you cross into a new region. So if five consecutive branches are all within the same 256-byte chunk, only one entry gets recorded. You're tracking region transitions, not individual branches. This naturally compresses long sequences of nearby branches into a compact history

- **ACYCLIC history:** This tries to get the repititive loop structure from the history, keeping only the most recent outcome of each branch address rather than the full sequence of outcomes. This is useful because in many cases, the most recent outcome of a branch is more predictive than the full history, especially in loops where the same branches are executed multiple times.

- **Forward Branch IMLI:** Usually IMLI is designed for backward branches, but the author has added a version for *top testing* loops i.e. where the exit is at the top and the unconditional jump is at the bottom. This is a very common optimization that compilers do (check here for reference).

3.1.1 Branch Misalignment and MODHIST

Consider the following code snippet:

```
if (x > 0) {
    if (x < 100) {      % the potentially misaligning branch
        do_something();
    }
}
```

When `x > 0` and `x < 100`, both branches execute and both get recorded into the history register. When `x > 0` but `x >= 100`, only the outer branch executes. This means the inner branch sometimes shows up in history and sometimes does not, which causes a problem.

Suppose there are two other branches later in the code, call them B and C, that always execute after this block. When the inner branch ran, B and C get pushed one slot further back in the history register. When it did not run, B and C sit one slot closer to the front. Same program, same behavior from B and C, but they appear at different positions in history depending on whether that inner branch fired.

TAGE hashes based on exact bit positions, so it sees these two situations as completely different and stores them in different table entries. Each entry gets trained half as often and the predictor never builds enough confidence to predict reliably. This is the misalignment problem.

MODHIST fixes this by simply not recording branches whose address does not satisfy:

```
branch_address mod N == 0
```

The idea is that guard branches like null checks and bounds checks, which are the typical culprits behind misalignment, tend to land at non-aligned addresses in compiled code. By filtering them out, B and C always appear at the same position in history, and the predictor can finally learn a consistent pattern.

3.2 MPP vs Baseline Predictor

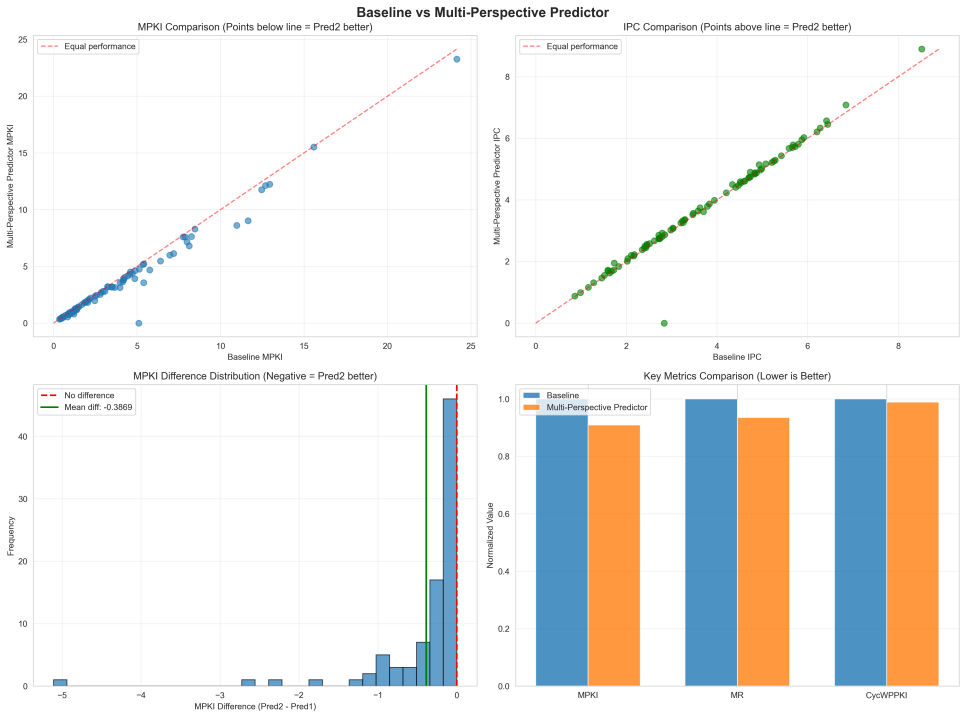

Figure 1: Comparison of MPP and Baseline Predictors

MPP proves to perform better than the baseline predictor, achieving a mean MPKI difference of 0.38 across all traces. This is a pretty decent improvement. One of the most interesting things

about the MPP is that it uses two bloom filters to identify branches that are trivially T or NT. Looking at the top 10 improvements, 6 of them are ***int_*** traces. As mentioned before, integer code is compiled from high level languages and has complex control flow, and compilers usually generate *alignment breaking branches* as cited in the paper,

> *"A branch branches over another branch. The second branch sometimes appears in the branch history and sometimes does not appear, leading to the same branch outcomes appearing in different locations in the global history."*

These branches appear as null checks, type checks and bound checks. This is where TAGE's strength becomes an issue i.e. a single misaligned branch "poisons" a long history entry. The MODHIST as mentioned above aims to solve this particular problem, and it seems to be doing a good job at it.

Table 1: Framework-Level Summary: MPP vs Baseline Predictor

Framework	Improve/Total	Success Ratio (%)	Total MPKI Improvement
Web	26/26	100.0	-10.4414
Int	28/37	75.7	-18.8957
Fp	8/14	57.1	-2.3088
Media	3/4	75.0	-0.5673
Infra	5/16	31.3	-1.3662
Compress	1/8	12.5	-0.9672

The improvement in ***Web_*** traces is not as dramatic but is remarkably consistent, having a 100% success rate. The precise path history of web traces becomes too specific, i.e. two slightly different web page loads produce completely different path histories even though their branch behaviour is very similar. The MPP's use of the BLURRYPATH feature allows it to generalize better and thus achieve a consistent improvement across all web traces.

The compress and infra traces are interesting because they show very little improvement over the baseline. For the compress traces I think the explanation is fairly simple. Their baseline MPKI is already very low, so TAGE-SC-L is already doing a good job and there is not much left for MPP to improve. The infra traces are honestly less clear to me. My best guess is that these workloads contain branches whose outcomes depend more on runtime data values than on control flow history, in which case none of MPP's history-based features would help. It is also possible that MODHIST's filtering is actually discarding branches that carry useful signal in these specific workloads, which would work against MPP rather than helping it. That said, I want to be clear that these are speculative explanations. Without knowing exactly what workloads these traces represent, it is difficult to say with confidence why MPP struggles to improve them.

4 Bullseye Predictor

4.1 Details

The bullseye predictor is driven by a core observation, stating that most of the mispredictions in TAGE-SC-L come from a small number of "hard-to-predict" branches. The bullseye predictor is designed to identify these hard-to-predict branches and apply a specialized prediction strategy to them, while leaving the rest of the branches to be handled by a more traditional predictor. H2P branches appear under different global histories every time they execute. For a predictor like TAGE that relies on global history, this is a very difficult pattern to learn. The paper also

notes that simply making TAGE bigger doesn't fix this problem, which means even an infinitely large TAGE would barely improve the prediction rates. The main problem isn't storage, but the branch's behaviour not correlating well enough with the global history.

The bullseye predictor adds a small 256k "subsystem" on top of the TAGE-SC-L predictor. It is composed of 3 main components:

- **H2P (Hard-to-Predict) Branch Classifier:** This component is responsible for identifying the hard-to-predict branches. This tracks every branch's execution count, MP count and running accuracy (the number of correct predictions divided by the total number of predictions for that branch). When a branch's execution count exceeds a certain threshold and its accuracy falls below a certain number, the classifier flags the branch as an H2P branch, preventing the entire system from being overwhelmed by the noise of these branches and allowing it to focus on the branches that it can predict well. In practice no more than 10 branches are active at once.

- **Two Perceptron Predictors:** Bullseye predictor uses two perceptron predictors, both of them running in parallel with the TAGE-SC-L predictor. One of them uses local history and the other uses folded global history to essentially "catch" long-range correlation that TAGE-SC-L misses. These perceptrons dedicate their weights to one hard branch rather than sharing tables across and with thousands of branches (something that TAGE does).

- **Trial phase and arbitration:** A newly flagged branch gets 512 executions to warm up its weights. After that the system compares which predictor is winning (like a tournament predictor). There is a confidence counter that reinforces the winning predictor and allows it to take over the prediction of that branch. The TAGE is set to default, but as soon as the perceptron wins 128 consecutive predictions, the TAGE updates are suppressed, stopping the thrashing cycle (where TAGE and the perceptron keep taking over the prediction of the branch and thus never giving either of them enough time to learn the pattern) once and for all.